IndicNeuroSym: Neuro-Symbolic Constrained Decoding for Classical Dravidian Metrical Poetry

Telugu *Dvipada* and Kannada *Ragale*

Samvaran K. Rallabandi, Mahesh Emani,
Pradeep Pachipulusu, and **Faviana Noronha**
IIIT Hyderabad
{samvaran.rallabandi, mahesh.emani}@research.iiit.ac.in
{saipradeep.p, faviana.noronha}@students.iiit.ac.in

Abstract

We present IndicNeuroSym, a neuro-symbolic pipeline for generating metrically valid classical Dravidian poetry in two metres: Telugu *dvipada* and Kannada *utsaha ragale*. Both forms enforce strict prosodic rules over syllable weight (*guru*/*laghu*), metrical foot patterns (*gaṇa*), and second-syllable rhyme (*prāsa*); *dvipada* additionally requires caesura alliteration (*yati*). We curate a validated corpus of 27,881 *dvipada* couplets from five classical Telugu sources together with 1,010 synthetic Kannada *ragale* poems, achieving 100% prosodic integrity by construction via a custom chandas scanner. We fine-tune Gemma 3 1B with LoRA on instruction-tuned pairs and pair it with a constrained decoder that intercepts token logits through a chained Finite State Transducer (FST) pipeline feeding parallel Non-deterministic Finite Automata (NFAs) for each prosodic constraint. We benchmark twelve configurations (two models $\times$ three decoding strategies $\times$ two languages, 1,224 generations total): the hybrid strategy—NFA alive-predicate masking plus a second accept-state re-ranking pass over the masked top-100—reaches **100% poem accuracy** on both metres with the fine-tuned model and on *dvipada* even with the base model. LoRA fine-tuning lifts Kannada masking-only validity from 12.7% to 96.1%, demonstrating that adapter training collapses most of the symbolic search burden into the model prior. As an external comparison we port the top-k constrained decoding algorithm of Chandomitra (Jagadeeshan et al., 2026) (the closest published constraint-decoding system for Indic metrical poetry, originally targeting Sanskrit Anuṣṭubh) to the Dvipada rule set; on the same models and seeds the adapted port reaches 1.0% / 13.7% poem accuracy, against which our hybrid achieves 7.3$\times$ higher accuracy and nearly 40$\times$ lower per-poem latency on the merged model.

1 Introduction

Classical Telugu literature contains a rich tradition of structured poetic forms. Among these, *dvipada* (literally "two feet") is one of the oldest metres, forming the backbone of major epics such as *Ranganatha Ramayanamu* (13th century) and *Basavapuranamu*. A *dvipada* couplet consists of exactly two lines (*pādalu*), each composed of four metrical feet (*gaṇalu*): three Indra-class feet followed by one Surya-class foot. Additional constraints govern second-syllable consonant rhyme (*prāsa*) and alliterative harmony between designated feet within a line (*yati*).

Generating metrically valid classical poetry in these languages presents three core challenges: (i) standard subword tokenizers (including the SentencePiece tokenizer used by Gemma 3) fragment *akṣaras* (syllables) across prosodic boundaries (Sennrich et al., 2016); (ii) the metrical constraints are tight in both traditions, with Telugu *dvipada* simultaneously enforcing *gaṇa*, *prāsa*, and *yati*, and Kannada *utsaha ragale* demanding strict *prāsa*, fixed 3+3+3+*guru gaṇa* alignments, and a ban on iambic (*laghu–guru*) sequences within any line (*pāda*); and (iii) annotated training corpora for these classical Dravidian metrical forms are exceptionally scarce.

This paper makes four contributions: (1) a curated, rigorously validated corpus of 27,881 *dvipada* couplets with Telugu and English meanings, complemented by 1,010 synthetic Kannada *ragale* poems at 100% prosodic integrity by construction; (2) an instruction fine-tuning (IFT) dataset and training pipeline that produces a per-language LoRA adapter for Gemma 3 1B; (3) a neuro-symbolic constrained decoding architecture pairing a chained Finite State Transducer (FST) pipeline with parallel Non-deterministic Finite Automata (NFAs), one per prosodic constraint, that guarantees metrical

validity by construction; and (4) an end-to-end benchmark of 1,224 constrained generations across twelve configurations (two models × three decoding strategies × two languages), plus an external head-to-head comparison on Dvipada against our port of Chandomitra's top-k constrained-decoding algorithm (Jagadeeshan et al., 2026), all positioned within the Generation and Analysis taxonomy of Kranti and Vajjala (2024).

2 Related Work

Constrained text generation. Constrained decoding frameworks enforce formal constraints during token generation without model retraining. Outlines (Willard and Louf, 2023) uses FSMs for JSON and regex constraints; LMQL (Beurer-Kellner et al., 2023) provides a query language for structured generation; and Guidance (Guidance-AI, 2023) implements grammar-constrained decoding via token masking. Our work extends this line to Telugu and Kannada prosodic metres, where constraints derive from classical chandas treatises.

Multilingual sentence embeddings. Cross-lingual alignment is evaluated using LaBSE (Feng et al., 2022), while intra-lingual poem-to-meaning similarity is assessed via mSBERT (Reimers and Gurevych, 2020). To provide a specialized comparison for Indic languages, L3Cube-IndicSBERT (Deode et al., 2023) is incorporated into the analysis. Furthermore, the LLM-as-a-Judge evaluation protocol is modeled after the G-Eval (Liu et al., 2023) and GEMBA (Kocmi and Federmann, 2023) metrics.

Computational prosody for Indic languages. Prior work on Indic metre analysis has focused primarily on Sanskrit; the closest precedent is Chandomitra (Jagadeeshan et al., 2026), a constraint-decoding system for Sanskrit Anuṣṭubh poetry. At each generation step Chandomitra's Algorithm 1 samples the top-k candidate tokens, computes the resulting syllable-weight (*laghu*/*guru*) sequence, and matches it against pre-compiled *fixed-length regex filters* per *pāda* (e.g., `^.{4}lgg.$` for the first *pāda* of Anuṣṭubh); if no candidate matches, k is widened and the filter is re-applied. This works well on Anuṣṭubh, where every *pāda* is exactly eight syllables and a single fixed regex fully specifies its valid shape (Jagadeeshan et al. report 99.86% syntactic accuracy on Sanskrit Anuṣṭubh with NLLB-distilled-1.3B fine-tuned on their corpus). The approach does *not* extend cleanly to Telugu *dvipada*, whose lines are variable-length (11–15 syllables) and built compositionally from a vocabulary of 6+2 *gaṇa* patterns: expressing the legal line shape of *dvipada* requires a regular language over foot patterns rather than a single fixed regex over syllable weights, and the metre adds *prāsa* and *yati* constraints absent from Anuṣṭubh entirely. We nonetheless port Chandomitra's Algorithm 1 to Dvipada (replacing the fixed-length per-*pāda* regex with a prefix trie of all 432 valid Indra^3·Surya foot sequences and adding separate *prāsa* and *yati* checks) and use it as an external baseline in Section 9, applied to the same models, prompts, and seeds as our enforcer. Telugu and Kannada-specific computational prosody otherwise remains largely unexplored, and to our knowledge this is the first study to combine formal *dvipada* and *ragale* prosody rules with neural language model generation for either language.

3 Prosodic Frameworks

Both metres operate at the syllable, foot, and line levels, with rules sourced from classical chandas treatises: *Chandhodarpanamu* (Chirravuri Srirama Murthy, 1998) for Telugu *dvipada* and Venkatachala Shastri (2008) for Kannada *utsaha ragale*. Complete rule tables appear in Appendix A.

3.1 Telugu *Dvipada*

A *dvipada* stanza contains exactly two lines (*pādalu*); each line comprises four metrical feet (*gaṇalu*) in the fixed sequence three Indra-class feet followed by one Surya-class foot. Each syllable is classified as heavy (*guru*, U) or light (*laghu*, I): long vowels, diphthongs, anusvara, visarga, syllable-final consonants, and the position before a conjunct cluster yield *guru*; short vowels yield *laghu* by default (full rules and the vocalic-*r* exception in Appendix A.1). Table 1 lists the eight permitted foot patterns (six Indra, two Surya). *Prāsa* requires that the base consonant of the second syllable of line 1 match that of line 2, subject to three phonemic equivalence groups (laterals {la,ḷa}, sibilants {śa,sa}, rhotics {ṛa,ra}; Appendix A.2). *Yati* requires that the first syllable

of foot 1 harmonically match the first syllable of foot 3 within a line, under 11 *maitri* equivalence groups (Appendix A.3). *Samanya dvipada* strictly enforces *prāsa*; *manjari dvipada* relaxes it.

Table 1: Permitted metrical feet (*gaṇalu*) for Telugu *dvipada*. U = *guru* (heavy), I = *laghu* (light).

Foot	Pattern	Class	Beats
nala	I I I I	Indra	4
naga	I I I U	Indra	5
sala	I I U I	Indra	5
bha	U I I	Indra	4
ra	U I U	Indra	5
ta	U U I	Indra	5
na	I I I	Surya	3
ha / gala	U I	Surya	3

3.2 Kannada *Utsaha Ragale*

A *utsaha ragale* stanza contains two lines, each comprising four metrical feet of exactly three syllables, yielding 12 syllables per line. Syllable weight follows the same broad rules as Dvipada (long vowels, diphthongs, anusvara, visarga, halant endings, and position before a conjunct cluster all promote to *guru*); the character sets differ only in script. Word boundaries block the conjunct rule. Table 2 lists the two permitted foot patterns: III at positions 1–3 and IIU at all four positions. The metre prohibits two-syllable patterns to preserve the triple-syllable rhythm, and requires both lines to end on *guru*, structurally enforced by constraining position 4 to IIU. *Ādi Prāsa* requires matching the base consonant of the second syllable across lines (ignoring vowel marks, conjunct extensions, anusvara, and visarga); relaxed evaluation allows two equivalence groups, laterals {la,ḷa} and sibilants {śa,ṣa,sa} (Appendix A.2).

Table 2: Permitted metrical feet (*gaṇalu*) for Kannada *utsaha ragale*. U = *guru* (heavy), I = *laghu* (light).

Pattern	Syllables	Allowed Positions	Beats
I I I	3	1, 2, 3	3
I I U	3	1, 2, 3, 4	4

4 Dataset Construction

4.1 Sources and Curation

To enable instruction-tuned generation in a metre with no public training corpus, we assemble a validated *dvipada* dataset from five classical Telugu sources and complement it with 1,010 synthetic Kannada *ragale* poems across 100 topics, generated with Gemini 3 Pro to mitigate data scarcity. Every candidate is filtered through a chandas analyzer so that the training distribution is 100% prosodically valid by construction. Table 3 reports raw couplet counts, the number passing strict 100% prosodic compliance (*pure*), and purity rates per source.

All candidates were filtered by the Dvipada and Ragale chandas analyzers, which verify full prosodic compliance and reject any couplet that fails *gaṇa*, *prāsa*, or *yati*. The high rejection rate for *Palanati Vira Charitra* (91.7%) reflects its use of the *manjari* variant, which relaxes *prāsa*. The final Telugu dataset comprises 27,881 couplets in JSON format, each paired with a Telugu prose meaning and an English translation. The Kannada *ragale* corpus follows the same schema.

Source	Raw Couplets	Master (Validated)	Purity
Ranganatha Ramayanamu	26,296	21,828	83.0%
Basavapuranamu	2,454	1,859	75.8%
Dvipada Bhagavatamu	3,157	2,002	63.4%
Palanati Vira Charitra	783	65	8.3%
Srirama Parinayamu	392	374	95.4%
Synthetic (Gemini)	3,496	1,753	50.1%
Total	**36,578**	**27,881**	**76.2%**

Table 3: Dataset sources with raw couplet counts, master validated counts, and prosodic purity rates.

4.2 Dataset Composition

The Telugu corpus is 93.7% classical material (26,128 couplets) and 6.3% synthetic (1,753 couplets). Topics span theology, mythology, and devotional poetry, primarily in archaic Telugu register with colloquial augmentations in the synthetic subset. The Kannada *ragale* corpus is 100% synthetic, covering 100 topics across generic topics.

5 Data Validation

We certify the 27,881-couplet corpus with a five-level pipeline that ascends from hard structural gates (Levels 1–2: prosody, prose-verse alignment) through diagnostic semantic and lexical checks (Levels 3–4) to a qualitative LLM-judge (Level 5); each level catches what the previous one cannot. Table 4 summarises pass rates.

5.1 Level 1: Prosodic Integrity

The chandas scanner serves as the inclusion criterion described in Section 4, so the trivially necessary first check is that every retained couplet

passes it: 100% (27,881/27,881). This is by construction, but it pins the lower bound for everything that follows.

5.2 Level 2: Length Ratio Check

A poem can be metrically valid yet paired with a prose meaning that is either too terse or too verbose to be a faithful gloss. We enforce $0.8 \leq |P|/|V| \leq 2.5$ (Telugu prose P over verse V), which 99.1% of records pass (27,633/27,881). The bulk (88.9%) sits in the 1.0–2.0 band; only 248 records (0.9%) fall outside, with the five most extreme ratios at 5.11, 4.73, 4.71, 4.65, and 4.36, all suspect translations rather than valid couplets.

5.3 Level 3: Semantic Fidelity

Levels 1–2 check structure but not meaning, so we next quantify semantic agreement using three multilingual sentence-embedding models, computed on the augmented dataset ($n = 29{,}343$) prior to final purity filtering. The primary gate is LaBSE Telugu-to-English meaning similarity ≥ 0.65 (Feng et al., 2022), which 77.1% of pairs clear (mean 0.721), confirming that prose annotations are translation-consistent. The other two scores (mSBERT-mpnet (Reimers and Gurevych, 2020) at 63.7% on poem-to-Telugu, L3Cube-IndicSBERT (Deode et al., 2023) at 40.1%) are intentionally diagnostic rather than gating: classical *dvipada* uses compressed, elliptical *samāsa* compounds that diverge structurally from modern prose paraphrases, so low poem-to-meaning similarities are expected and reflect genre distance, not annotation noise. The full per-pair table is in Appendix C.

5.4 Level 4: Lexical Diversity

Even with structure and meaning verified, the corpus could still be formulaic. Level 4 therefore characterises its literary register. The Gemma 3 tokenizer covers 74.4% of the model's Telugu vocabulary (1,328/1,784 tokens) on 804,671 corpus tokens. At the word level, corpus TTR is 0.491 and MATTR is 0.501, below the 0.70–0.80 range typical of modern prose, consistent with the formulaic repetition inherent to metrical verse. Yule's $K = 3.50$ places the corpus firmly in the literary range ($K < 100$), while Honoré's $H = 6{,}165$ far exceeds the 1,000–2,000 literary baseline, indicating exceptional vocabulary richness. The hapax ratio ($V_1/V = 0.804$) confirms that most word types occur exactly once. No exact duplicates were found; 160 poems in 79 groups are near-duplicates under normalisation. Full per-poem and corpus tables are in Table 18, Appendix C.

Table 4: Cross-level validation summary.

Level	Description	Pass Rate
1	Prosodic integrity (chandas)	100.0%
2	Length ratio (0.8–2.5)	99.1%
3	LaBSE semantic fidelity	77.1%
4	Lexical diversity (TTR)	0.90 avg
5	LLM-as-a-Judge (n=250)	97.0%
Pass all levels 1–3		68.9%

5.5 Level 5: LLM-as-a-Judge Evaluation

The first four levels are automatic and rule-based. Level 5 layers a qualitative judgement on top by sampling 250 couplets stratified across all sources and scoring them with Gemini 3 Pro (Vertex AI Batch API) on a 1–5 rubric covering semantic fidelity, completeness, cultural accuracy, and linguistic quality in Telugu and English (modelled on G-Eval (Liu et al., 2023) and GEMBA (Kocmi and Federmann, 2023)). The mean total is **24.25/25** (97.0%), with per-dimension means ranging from 4.73 (semantic fidelity) to 4.98 (Telugu linguistic quality); 90.4% of samples receive an overall verdict of "Excellent" (226/250), 6.4% "Good", 2.8% "Acceptable", and 0.4% "Poor" (1 sample). The per-dimension breakdown, aggregate distribution, and source-stratification appear in Appendix C. The five levels combined certify a corpus that is structurally correct, prose-verse aligned, semantically consistent, lexically rich, and qualitatively highly rated.

6 Fine-Tuning Dataset Preparation

The validated 27,881-entry corpus is transformed into instruction-completion pairs for supervised fine-tuning (SFT) of Gemma 3 1B (Gemma Team, 2024) using LoRA (Hu et al., 2022).

Instruction templates. Each sample is assigned one instruction type from four categories: (1) **Generic** ("Write a Telugu *dvipada* couplet following the above rules"), available for all samples; (2) **Work-style** ("Compose a *dvipada* in the style of [source]"), available for non-synthetic sources; (3) **Meaning-conditioned (Telugu)**: "Write a *dvipada* expressing: {Telugu meaning}"; and (4) **Meaning-conditioned (English)**: "Write a

dvipada expressing: {English meaning}". This diversity trains the model to respond to varied prompt styles in both languages.

Rules injection. Every training sample's input is prefixed with the complete *dvipada* prosodic specification, covering all *gaṇa* patterns, *guru/laghu* rules, *prāsa*, and *yati*. This ensures the model has the metrical specification in context during both training and inference.

Dataset splits. The pipeline produces `train.jsonl` (80%), `val.jsonl` (10%), and `test.jsonl` (10%, held out).

6.1 Training Setup

Table 5 reports the LoRA fine-tuning configuration used to produce the Dvipada merged model and the Ragale adapter benchmarked in Section 9. Both runs share the same base model (`google/gemma-3-1b-it`), optimizer (fused AdamW), schedule (cosine with warmup), precision (bf16), attention backend (SDPA), gradient checkpointing, and effective batch size 16 (via gradient accumulation), and both target every linear projection (`q,k,v,o,gate,up,down`) with BIAS=NONE and dropout 0.05. They diverge on LoRA capacity, learning rate, warmup, and sequence length, reflecting the $\sim$28$\times$ asymmetry between the Telugu and Kannada training corpora (27,881 vs. 1,010 examples).

Why the configurations differ. Three asymmetries are deliberate. First, the Kannada run uses a larger LoRA (r=32 vs. r=16) and a smaller learning rate (5e–5 vs. 2e–4) to extract a useful signal from the $\sim$1k-example Ragale corpus without overfitting; the higher warmup ratio (0.06) softens the early-step gradients on the same small data. Second, Ragale's longer max sequence length (512 vs. 384) accommodates its 12-syllable-per-line structure, which produces slightly longer instruction prompts than Dvipada's 11–15 syllable variable-length lines. Third, both runs save the best checkpoint by `eval_loss`; the Dvipada adapter is then merged back into the base weights to produce the `gemma3-1b-merged` model used in Section 9, while the Ragale adapter is retained as a PEFT layer on top of the base model (`gemma3-1b-lora`). This explains the model-naming convention used in Tables 6 and 7.

Table 5: LoRA fine-tuning configuration for the Dvipada merged model and the Ragale adapter. "All linear" denotes `q,k,v,o,gate,up,down` projection layers.

Hyperparameter	Telugu Dvipada	Kannada Ragale
Base model	`gemma-3-1b-it`	`gemma-3-1b-it`
LoRA rank r	**16**	**32**
LoRA α	32	64
LoRA dropout	0.05	0.05
Target modules	all linear	all linear
LoRA bias	none	none
Optimizer	AdamW (fused)	AdamW (fused)
Learning rate	**2e–4**	**5e–5**
LR schedule	cosine	cosine
Warmup ratio	0.03	0.06
Weight decay	0.01	0.01
Max grad. norm	1.0	1.0
Per-device batch	1	2
Grad. accum. steps	16	8
Effective batch	16	16
Max seq. length	384	512
Seq. packing	yes	yes
Epochs	**8**	**6**
Train / val split	90 / 10	90 / 10
Best-ckpt criterion	min eval_loss	min eval_loss
Eval / save every	500 steps	500 steps
Precision	bf16 + TF32	bf16
Attention	SDPA	SDPA
Grad. checkpointing	yes	yes
Hardware	RTX 5050 (8 GB)	RTX 5050 (8 GB)
Seed	42	42

7 Constrained Decoding Architecture

To provide formal metrical guarantees beyond what fine-tuning alone achieves, we design a constrained decoding system that intercepts Gemma's token generation and masks out any token violating *dvipada* metre.

7.1 Theoretical Basis

Every *dvipada* constraint operates on fixed positions with finite alphabets, so the combined constraint language is regular: the *gaṇa* pattern is a finite union of concatenations of finite strings over {U,I} (no stack required); *prāsa* is a single comparison at a fixed position against $\sim$30 stored consonant classes; *yati* is a single comparison at a fixed position against 11 *maitri* groups. Closure of regular languages under intersection keeps the combined machine an NFA of bounded size.

7.2 System Overview

At each generation step, raw logits from Gemma 3 1B pass through the Symbolic Enforcer (Figure 1, Appendix B) before sampling. The Enforcer routes candidate tokens through a chained FST pipeline into three parallel NFAs and emits a boolean mask ($-\infty$ for invalid tokens). The

decoder strategies described below in Section 7.7 consume this mask in three different ways (Figure 2, Appendix F).

7.3 Layer 1: Orchestrator FST Pipeline

Two chained Mealy FSTs transform raw Unicode token strings into a classified U/I syllable stream. A stateless character classifier precedes them.

Stage 0: Codepoint Classifier (stateless). A stateless classification function classifies each Unicode codepoint into 11 categories via an $O(1)$ set-membership check. Categories cover consonants (U+0C15–U+0C39), independent vowels (U+0C05–U+0C14), matras (U+0C3E–U+0C4C), virama (U+0C4D), anusvara (U+0C02), visarga (U+0C03), ai-length mark (U+0C56), space, newline, ZWNJ, and other. This function is inlined into the Syllable Assembler rather than implemented as a separate transducer.

Stage 1: Syllable Assembler (4 states). Groups the category stream into complete Telugu syllables using states IDLE, CONSONANT_CLUSTER, PENDING_VIRAMA, and VOWEL. A one-slot *prev_syllable* buffer enables retroactive pollu (syllable-final consonant) merging. The PENDING_VIRAMA state distinguishes conjunct consonants from pollu marks via lookahead.

Stage 2: Guru/Laghu Classifier (2 states). Classifies each syllable as U or I using five phonological rules from *Chandhodarpanamu* (Chirravuri Srirama Murthy, 1998), via states EMPTY and PENDING_I. A one-syllable delay buffer in PENDING_I enables Rule 5 (conjunct-induced weight promotion), which applies within words but is blocked by intervening spaces.

7.4 Layer 2: Position Tracker and Parallel NFAs

The Position Tracker routes syllable metadata to three NFAs based on the current position within the couplet: line index (1 or 2), syllable index within the line, and line completion count.

Gaṇa NFA (∼69 states). Validates each line against the regular pattern $(nala \mid naga \mid sala \mid bha \mid ra \mid ta)^3 \cdot (na \mid ha)$ over {U, I}. The NFA branches non-deterministically after each syllable. Each branch is a tuple (`slot`, `gana_name`, `sub_pos`, `matched_ganas`), where slot indexes the current foot (0–2 for Indra *gaṇas*, 3 for the Surya *gaṇa*). Valid line lengths range from 11 to 15 syllables.

Prāsa NFA (7 states). Phase 1 records the base consonant of the second syllable of line 1. Phase 2 verifies that the second syllable of line 2 has a matching base consonant (checked via equivalence lookup); a mismatch transitions to the REJECT state. The 7 states are: LINE1_SYL0, LINE1_SYL1, LINE1_REST, LINE2_SYL0, LINE2_SYL1, ACCEPT, and REJECT. Consonant identity is stored as a data variable, not encoded in NFA states. Three equivalence pairs (laterals: l/L, sibilants: sh/Sh/s, rhotics: R/r) are checked via lookup.

Yati NFA (4 states). The state IDLE records the phonetic information of the first syllable of *gaṇa* 1; RECORDED passes through *gaṇa* 2 silently; on entering *gaṇa* 3 the NFA verifies that its first syllable belongs to the same phonetic class via a 5-level cascade (exact match, *vyanjana yati*, *svara yati*, *samyukta yati*, *bindu yati*), transitioning to ACCEPTED on success or REJECTED on failure. Phonetic group membership (across 11 *Yati Maitri* groups) is computed on demand rather than encoded in NFA states. *Yati* enforcement is optional; it can be disabled for *manjari* variant generation.

7.5 Dead State Detection

To maintain metrical integrity, we define a *Dead State Detection* mechanism. Let $\mathcal{B}$ be the set of active NFA branches. For each branch $b \in \mathcal{B}$ at state $s = (\texttt{slot}, \texttt{gana}, \texttt{pos})$, let $C(b)$ be the current syllable count. We define two closed-form heuristic functions:

- $min_dist(s)$: Minimum syllables to reach ACCEPT from s.
- $max_dist(s)$: Maximum syllables to reach ACCEPT from s.

A branch b is considered **viable** if it satisfies the window constraint $[L_{min}, L_{max}]$, where $L_{min} = 11$ and $L_{max} = 15$:

$$(C(b) + \textit{min_dist}(s) \leq 15) \wedge (C(b) + \textit{max_dist}(s) \geq 11) \quad (1)$$

If the set of viable branches is $\emptyset$, the configuration is flagged as a *dead state*, triggering immediate token pruning.

7.6 State Budget and Design Rationale

Using three independent NFAs achieves a $24\times$ state reduction ($69 \times 7 \times 4 = 1{,}932$ product DFA states versus $\sim$80 total separately), while enabling partial scoring, selective constraint enforcement, and independent debugging. Per-component state counts appear in Table 13, Appendix B.

7.7 Constrained Decoding Loop

All three decoding strategies we benchmark rely on per-step NFA rejection: at every generation step, BUILDGANAMASK (Alg. 3) clones the incremental COMPOSITESTATE, feeds each of the $\sim$1,822 Telugu tokens, flushes, and keeps only those whose IS_ALIVE() predicate still holds—i.e., the token keeps the Gaṇa, Prāsa, and Yati NFAs jointly satisfiable. What distinguishes the strategies is (a) whether a recovery mechanism runs when the alive set collapses, and (b) whether the surviving candidates are further re-ranked by a line-completion (HAS_ACCEPT) preference. Full pseudocode is in Appendix F; for completeness the appendix also presents a stateless *Rejection Sampling* variant (Alg. 4) that runs ANALYZETEXT on the full text per candidate and enforces only gaṇa reachability, included as an illustrative alternative but not benchmarked in Section 9.

Strategy 1 (Masking-Only): a static mask first reduces the $\sim$256K vocabulary to $\sim$1,822 Telugu tokens, then BUILDGANAMASK keeps only those tokens whose IS_ALIVE() predicate holds; top-p sampling runs over this NFA-filtered set. No recovery, no accept-state re-ranking.

Strategy 2 (Masking + Backtracking, Alg. 5): same NFA alive-predicate filtering as Strategy 1, plus a recovery mechanism—if the alive set drops below 3 candidates or the line overshoots 15 syllables, the decoder restores a checkpoint (ring buffer of 15 entries, saved every 3 tokens) and escalates the sampling temperature by $+0.15$ (cap 1.2, decay over 6 steps).

Strategy 3 (Hybrid Mask + Accept Preference, Alg. 6): same NFA alive-predicate filtering as Strategy 1, plus a *second* NFA pass over the top-100 tokens of the masked distribution that additionally checks HAS_ACCEPT() per candidate and prefers tokens that would complete a valid line when weighted-sampling the next token. Checkpoint backtracking is retained as a last-resort fallback but rarely fires in practice (Section 9). The "rejection" in our original name for this strategy refers specifically to this second HAS_ACCEPT re-ranking pass, not to the underlying alive-predicate filtering that all three strategies share.

7.8 Ragale Adaptation

For Kannada *utsaha ragale* we apply the same Symbolic Enforcer architecture with three structural simplifications driven by the rule set of Venkatachala Shastri (2008): (i) the Gaṇa NFA shrinks from 69 to 22 states, since only two foot patterns (III and IIU) are permitted at fixed 12 syllables per line; (ii) the Yati NFA is removed entirely (*ragale* has no caesura constraint); and (iii) the Prāsa NFA uses two Kannada-specific equivalence groups instead of three (the Telugu rhotic group is absent). The total state budget falls from $\sim$80 to $\sim$35; the product DFA falls from 1,932 to 154 ($12.5\times$ smaller). All three decoding strategies apply to Ragale without algorithmic modification, only with the reduced *gaṇa* pool and the disabled yati check. Full construction details, the Dvipada-vs-Ragale comparison table, the Ragale state budget, and the architecture figure appear in Appendix G.

8 End-to-End Evaluation

We evaluate the symbolic enforcer end-to-end on both metres by running every combination of two models and three decoding strategies for twelve benchmark configurations. For each language we test the base instruction-tuned model (`google/gemma-3-1b-it`) and its LoRA fine-tuned variant (merged for Telugu, adapter for Kannada; full hyperparameters in Section 6.1, Table 5) under the three decoding strategies of Section 7: *masking only*, *masking with checkpoint backtracking*, and the *hybrid* mask + accept-state re-ranking sampler. Each (model, method) cell generates 102 poems from 3 prompts $\times$ 34 seeds ($42 + 7k$ for $k \in [0, 33]$), giving **1,224 generations** across the matrix. Sampling uses $\tau = 0.7$, top-$p = 0.9$, `max_new_tokens` $= 150$, on a single NVIDIA RTX 5050 laptop GPU with bfloat16 (no quantisation). Telugu prompts are mythological/devotional themes drawn from the corpus (*tallI prEma annintikaMTe goppadi*, *hanumaMtuDu samudramunu dATi laMkanu cErenu*, *SrIrAmuDu sItAdEvini rakshiMcuTaku laMkaku veLLenu*); Kannada prompts are short topical themes (*Bubbles*, *Moon*, *Rain*). We

Table 6: Dvipada end-to-end constrained decoding results ($n = 102$ poems per row; 3 prompts $\times$ 34 seeds). The *adapted Chandomitra* rows are our port of the Algorithm 1 of Jagadeeshan et al. (2026) (originally designed for Sanskrit Anuṣṭubh, 99.86% on its native task) to the Dvipada rule set, applied to the same models, prompts, and seeds. All three FST+NFA strategies share the same per-step NFA alive-predicate filter; they differ only in recovery (backtracking) and accept-state re-ranking (hybrid).

Model	Method	Valid	Poem %	Line %	s/poem
Adapted Chandomitra baseline (Jagadeeshan et al., 2026)					
gemma3-1b-base	chandomitra	1/102	1.0	3.4	37.24
gemma3-1b-merged	chandomitra	14/102	13.7	18.1	58.87
FST+NFA Symbolic Enforcer (this work)					
gemma3-1b-base	masking_only	32/102	31.4	61.8	3.49
gemma3-1b-base	masking_backtrack	93/102	91.2	95.6	3.45
gemma3-1b-base	hybrid	102/102	**100.0**	100.0	1.99
gemma3-1b-merged	masking_only	37/102	36.3	67.6	2.10
gemma3-1b-merged	masking_backtrack	80/102	78.4	89.2	2.65
gemma3-1b-merged	hybrid	102/102	**100.0**	100.0	1.53

Table 7: Ragale end-to-end constrained decoding results ($n = 102$ poems per row; 3 prompts $\times$ 34 seeds). All three strategies share the same per-step NFA alive-predicate filter; hybrid additionally applies an accept-state re-ranking pass over the masked top-100.

Model	Method	Valid	Poem %	Line %	s/poem
gemma3-1b-base	masking_only	13/102	12.7	37.0	10.71
gemma3-1b-base	masking_backtrack	13/102	12.7	37.0	3.78
gemma3-1b-base	hybrid	69/102	67.6	80.2	1.33
gemma3-1b-lora	masking_only	98/102	96.1	96.6	1.42
gemma3-1b-lora	masking_backtrack	98/102	96.1	96.6	1.43
gemma3-1b-lora	hybrid	102/102	**100.0**	100.0	2.03

report *poem accuracy* (fraction of generations whose both lines pass the FST+NFA validator), *line accuracy*, mean wall time per poem, mean tokens generated, and mean backtrack count; validation uses the same chandas pipeline that filtered the training corpus. All twelve benchmark scripts and result JSONs are distributed under `domino/` (Telugu) and `ragale_pipeline/ragale_inference_scripts/` (Kannada) in the project repository.

9 Results

Tables 6 and 7 report poem-level accuracy for the twelve configurations. Per-topic breakdowns and efficiency statistics are deferred to Appendix D.

Strategy comparison and feature-ladder ablation. Tables 6 and 7 also function as a feature-ladder ablation over the three strategies defined in Section 7.7. All three strategies share the same NFA alive-predicate filtering via BUILDGANAMASK at every step. *masking_only* provides only that shared filter. *masking_backtrack* adds checkpoint rollback with temperature escalation when the alive set collapses. *hybrid* instead adds a second NFA pass over the top-K_r masked candidates that re-ranks by HAS_ACCEPT (line-completion) preference — i.e., among surviving alive-tokens, it prefers those whose syllable count would actually complete a valid line. Hybrid is the only strategy that reaches 100% poem accuracy on every fine-tuned configuration (Dvipada base, Dvipada merged, Ragale-LoRA) and additionally on Dvipada-base, confirming the design intent: the alive-predicate filter alone only guarantees that the *next* token keeps the NFA reachable, whereas the accept-state re-ranking actively steers the decoder toward tokens that complete the line. On Dvipada-base, adding backtracking lifts validity $31.4\% \rightarrow 91.2\%$ ($2.9\times$); adding accept-state re-ranking on top of the same alive-filter closes the remaining nine points to 100%. On Ragale-base, backtracking contributes nothing ($12.7\% \rightarrow 12.7\%$) because the constrained alphabet leaves few alternative completions, and only accept-state re-ranking yields a substantial gain ($12.7\% \rightarrow 67.6\%$). This is not a controlled feature toggle (each strategy is a distinct decoder, not the previous one with one feature disabled), but the gradient of accuracies isolates the marginal value of each addition as the underlying constraint structure varies between metres. The per-topic breakdowns in Tables 22 and 23 show the hybrid wins are uniform across prompts, not driven by a single easy topic.

Effect of fine-tuning. The two languages reveal very different fine-tuning sensitivities. For Ragale, LoRA dramatically improves the simplest strategy (masking_only: $12.7\% \rightarrow 96.1\%$, a $7.6\times$ gain), bringing it within four points of full validity without any recovery or accept-state re-ranking. For Dvipada the same comparison is much smaller ($31.4\% \rightarrow 36.3\%$), but the merged model still reaches 100% under hybrid with the lowest wall time of any configuration (1.53 s/poem). Fine-tuning therefore collapses most of the search burden into the model's prior, leaving the decoder mostly to verify rather than steer.

Backtracking is asymmetric across languages. For Dvipada-base, checkpoint backtracking lifts validity from 31.4% to 91.2% (a $2.9\times$ gain), confirming that the larger Dvipada *gaṇa* space (8 patterns, 11–15 syllables) admits enough alternative completions for retry to succeed. For Ragale, masking_only and masking_backtrack

produce *identical* validity numbers (12.7% on base, 96.1% on LoRA); backtracking only changes runtime (Ragale-base: 1093 → 386 s, since backtracking escapes earlier from doomed sequences). This is consistent with the smaller Ragale *gaṇa* space (2 patterns, fixed 12 syllables): once the model drifts off, the constrained alphabet leaves no alternative path, so backtracking cannot find a different valid completion. Notably, on Dvipada-merged, masking_backtrack reaches only 78.4% while the same model under hybrid reaches 100%: checkpoint backtracking re-rolls the model into the same low-probability tail it just escaped from, whereas hybrid's accept-state re-ranking actively steers toward tokens that complete a valid line rather than merely keeping the NFA reachable.

Cost of constraints. Average tokens per poem decreases monotonically with constraint strength on the Dvipada base model (52.1 → 34.0 → 29.9 across the three methods), and hybrid has the lowest wall time on the LoRA models despite doing the most per-token work. Stronger constraints commit the model to valid syllables earlier, eliminating the wasted generation that the weaker strategies spend exploring dead ends. Appendix Table 24 reports the full per-poem efficiency profile.

Comparison to Chandomitra. Table 6 also reports an external constraint-decoding baseline: a faithful port of Chandomitra's Algorithm 1 (Jagadeeshan et al., 2026) to the Dvipada rule set.[1] Applied to the same models, prompts, and seeds as our enforcer, the adapted Chandomitra reaches only 1.0% poem accuracy on gemma3-1b-base and 13.7% on gemma3-1b-merged, compared to our masking_only (31.4% / 36.3%) and hybrid (100% / 100%). On the merged model the FST+NFA hybrid is 7.3× more accurate than the adapted Chandomitra and nearly 40× faster per poem (1.53 s vs 58.87 s in Table 24). The gap is consistent with a fundamental algorithmic limitation of the top-k-filter approach when transferred from Anuṣṭubh to Dvipada: Chandomitra-style decoding has no incremental state, no backtracking, and no dead-state pruning, so once a partial line drifts into a region where no valid continuation exists, the relaxation cascade has no way to recover and the bad token is committed permanently. Dvipada's variable 11–15 syllable lines composed from a vocabulary of foot patterns make such doomed prefixes far more common than Anuṣṭubh's fixed 8-syllable-per-*pāda* structure, where the entire line shape is known in advance. Our enforcer addresses this with an incremental COMPOSITESTATE (no full re-analysis per token), NFA alive-predicate filtering over the entire ∼1,822-token Telugu vocabulary at every step (rather than only the top-k as in Chandomitra-style decoding), dead-state pruning via gana reachability, and—in the hybrid variant—a second HAS_ACCEPT re-ranking pass over the masked top-100 that actively prefers line-completing tokens. We do not claim a controlled ablation of these features individually; the comparison is system-versus-system. We restrict the adapted-Chandomitra comparison to Dvipada because the published Chandomitra implementation handles only Devanagari Anuṣṭubh, and porting it to Kannada *ragale* would require both a Kannada syllable-weight analyzer and a new family of *ragale*-specific filters.

Failure modes. Ragale-base failures concentrate in two patterns: garbled near-Kannada output assembled from low-probability subword tokens that fail the syllable-count check, and short topic-echoing fragments (e.g. a single phrase derived from the English prompt) that satisfy neither *gaṇa* structure nor *prāsa*. Both arise because the base model has not internalised *ragale* structure from pretraining and tends to fall back on prompt-echoing under the constrained sampler. Fine-tuning eliminates this mode entirely, which explains the steep 12.7% → 96.1% jump under masking_only.

Limitations. Five caveats apply. (i) The Kannada *ragale* corpus is small (1,010 synthetic poems on 100 topics generated with Gemini 3 Pro); we have no classical *ragale* sources, so the LoRA adapter is trained entirely on synthetic data. (ii) The LLM-as-Judge evaluation in Section 5 uses Gemini 3 Pro to score data partly produced by Gemini 3 Pro itself, introducing a self-evaluation bias we cannot quantify without scholar annotation, a gap we plan to close in future work. (iii) All experiments use a

[1] The published Chandomitra system targets Sanskrit Anuṣṭubh (99.86% syntactic accuracy on its native task). Our port preserves the outer top-k masking loop verbatim, replaces the fixed-length per-*pāda* regex with a prefix trie of all 432 valid Indra^3·Surya foot sequences, and adds the *prāsa* and *yati* checks that Anuṣṭubh does not require. We refer to this port as "adapted Chandomitra" below.

single base model (Gemma 3 1B) on a single hardware configuration (RTX 5050 laptop, 8 GB VRAM); generalisation to larger models or other architectures remains an empirical hypothesis. (iv) The FST+NFA enforcer is restricted to regular-language constraints; metres requiring context-sensitive features (e.g., morpheme-level Sanskrit *vṛtta*) would need a more expressive machinery. (v) Our adapted Chandomitra baseline is a port of Algorithm 1 to Dvipada, not the published Sanskrit system; a cross-architecture comparison against published Chandomitra on its native task is out of scope.

Ethical considerations. The classical Telugu sources we use (*Ranganatha Ramayanamu*, *Basavapuranamu*, *Dvipada Bhagavatamu*, *Palanati Vira Charitra*, *Srirama Parinayamu*) are all public-domain works dating from the 13th–17th centuries. The synthetic *ragale* corpus uses deliberately neutral topics (Bubbles, Moon, Rain) to avoid generating doctrinal or sectarian content. Our LLM-as-Judge protocol involves no human subjects beyond the authors. The principal dual-use risk is that a fluent generator of devotional metrical poetry could be used to fabricate attributed scripture; we recommend any downstream deployment include explicit provenance markers and a clear "machine-generated" disclosure. Compute footprint is modest: all training and inference runs use a single laptop-class GPU.

10 Conclusion

We have presented IndicNeuroSym, a complete neuro-symbolic pipeline for classical Dravidian metrical poetry generation, instantiated on Telugu *dvipada* and Kannada *utsaha ragale*. Our four contributions are: (i) a 27,881-couplet validated Dvipada corpus and 1,010 synthetic Ragale poems, both at 100% prosodic integrity by construction, with a 97.0% mean LLM-judge score on a 250-couplet stratified sample; (ii) an instruction fine-tuning pipeline that produces a LoRA adapter per language; (iii) a chained FST + parallel NFA decoder using $\sim$80 states for Dvipada and $\sim$35 for Ragale, a $24\times$ and $4.4\times$ reduction over the corresponding product automata; and (iv) 1,224 end-to-end constrained generations across 12 configurations benchmarking three decoding strategies on two models per language, together with an external head-to-head comparison on Dvipada against our port of the Chandomitra top-k constrained-decoding algorithm (Jagadeeshan et al., 2026).

The headline finding is that the hybrid strategy—NFA alive-predicate masking plus a second accept-state re-ranking pass over the masked top-100—reaches **100% poem accuracy** on Dvipada with both base and merged models and on Ragale with the LoRA model, while being the *fastest* strategy on every fine-tuned configuration. On the same models and seeds the adapted Chandomitra port reaches only 1.0% / 13.7% on Dvipada, reflecting an algorithmic limitation of stateless top-k filtering on Dvipada's variable-length *gaṇa* composition, which lacks the fixed-shape line structure that makes the original algorithm work on Sanskrit Anuṣṭubh. LoRA fine-tuning lifts Ragale masking-only from 12.7% to 96.1%, demonstrating that even modest adapter training collapses most of the symbolic search burden into the model prior. The same architecture transfers between the two languages with only structural simplifications (no Yati NFA, smaller Gana NFA), no algorithmic changes to the three decoding strategies in Appendix F.

Future work. Six directions follow naturally: (i) expert validation by classical Telugu and Kannada scholars to refine the LLM-judge rubric and yield a human-validated gold subset; (ii) broader classical-corpus curation across additional Telugu metres under the same five-level pipeline; (iii) language-agnostic packaging that exposes the FST+NFA pipeline's Unicode and NFA-topology layers as configurable modules for new scripts; (iv) an *akṣara*-aware tokenizer to reduce the dynamic mask burden and improve fluency; (v) extension to additional metres (Kannada *ṣaṭpadi*, Telugu *sīsa*, Hindi/Marathi *chand*); and (vi) lexical-resource integration (thesauri and inflected verb-form tables) so masking prefers semantically and morphologically appropriate continuations.

References

C. Kranti and S. Vajjala, "METRICALARGS: A Taxonomy for Studying Metrical Poetry with LLMs," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

Manoj Balaji Jagadeeshan, Samarth Bhatia, Pretam Ray, Harshul Raj Surana, Akhil Rajeev P, Priya Mishra, Annarao Kulkarni, Ganesh Ramakrishnan,

Prathosh AP, and Pawan Goyal, "Chandomitra: Towards Generating Structured Sanskrit Poetry from Natural Language Inputs," *arXiv preprint arXiv:2506.00815*, 2026.

Fangxiaoyu Feng, Yinfei Yang, Daniel Cer, Naveen Arivazhagan, and Wei Wang, "Language-Agnostic BERT Sentence Embedding," *arXiv preprint arXiv:2007.01852*, 2022.

Nils Reimers and Iryna Gurevych. 2020. Making Monolingual Sentence Embeddings Multilingual Using Knowledge Distillation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4512–4525.

Samruddhi Deode, Janhavi Gadre, Aditi Kajale, Ananya Joshi, and Raviraj Joshi, "L3Cube-IndicSBERT: A simple approach for learning cross-lingual sentence representations using multilingual BERT," *arXiv preprint arXiv:2304.11434*, 2023.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu, "G-Eval: NLG Evaluation Using GPT-4 with Better Human Alignment," *arXiv preprint arXiv:2303.16634*, 2023.

Tom Kocmi and Christian Federmann, "Large Language Models Are State-of-the-Art Evaluators of Translation Quality," *Proceedings of the 24th Annual Conference of the European Association for Machine Translation*, pp. 193–203, 2023.

Brandon T. Willard and Rémi Louf, "Efficient Guided Generation for Large Language Models," *arXiv preprint arXiv:2307.09702*, 2023.

Luca Beurer-Kellner, Marc Fischer, and Martin Vechev, "Prompting Is Programming: A Query Language for Large Language Models," *Proceedings of the ACM on Programming Languages (PLDI)*, vol. 7, 2023.

Guidance-AI, "Guidance: A Guidance Language for Controlling Large Language Models," https://github.com/guidance-ai/guidance, 2023.

Sreenivas Nidamarthy, "Comparative Study of Kannada and Telugu Consonants, Consonant Sub Forms, Vowels, Signs," *Shikshan Sanshodhan: Journal of Arts, Humanities and Social Sciences*, vol. 4, no. 9, 2021.

Chirravuri Srirama Murthy, "Chandodarpanamu," Digitised text at https://archive.org/details/chandodarpanamu018848mbp, 1848.

T. V. Venkatachala Shastri, *Kannada Chandaswaroopa*, DVK Murthy Publication, Mysore, 3rd edition, 2008, pp. 267–315.

Gemma Team, "Gemma: Open Models Based on Gemini Research and Technology," *arXiv preprint arXiv:2403.08295*, 2024.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen, "LoRA: Low-Rank Adaptation of Large Language Models," *International Conference on Learning Representations (ICLR)*, 2022.

Rico Sennrich, Barry Haddow, and Alexandra Birch, "Neural Machine Translation of Rare Words with Subword Units," *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 1715–1725, 2016.

Mildred C. Templin. 1957. *Certain Language Skills in Children: Their Development and Interrelationships*. University of Minnesota Press, Minneapolis.

Michael A. Covington and Joe D. McFall. 2010. Cutting the Gordian Knot: The Moving-Average Type–Token Ratio (MATTR). *Journal of Quantitative Linguistics*, 17(2):94–100.

George Udny Yule. 1944. *The Statistical Study of Literary Vocabulary*. Cambridge University Press.

Anthony Honoré. 1979. Some Simple Measures of Richness of Vocabulary. *Association for Literary and Linguistic Computing (ALLC) Bulletin*, 7(2):172–177.

Herbert S. Sichel. 1975. On a Distribution Law for Word Frequencies. *Journal of the American Statistical Association*, 70(351):542–547.

A Complete *Dvipada* Prosodic Rules

This appendix provides complete rule tables for the *dvipada* prosodic framework summarised in Section 3.

A.1 Guru-Laghu Classification Rules

Table 8 lists the five conditions under which a syllable is classified as *guru* (U). A syllable not meeting any condition defaults to *laghu* (I). Rule 5 is subject to the word-boundary constraint: it fires when the next syllable begins a conjunct cluster *within the same word* but is blocked by an intervening space; newlines are transparent.

Table 8: Guru-laghu classification rules from *Chandhodarpanamu* (Chirravuri Srirama Murthy, 1998).

R.	Name	Condition
1	Long vowel	Syllable contains aa, ii, uu, ee, oo, ai, or au
2	Diphthong	Vowel is ai or au
3	Anusvara / Visarga	Syllable ends in nasalisation (ṃ) or aspiration (h)
4	Pollu	Syllable ends in a virama (bare consonant, no following vowel)
5	Conjunct position	Next syllable (same word) begins with a conjunct cluster

Exceptions retained as laghu:
(a) Syllables containing the vocalic ṛ/Ṝ and the immediately preceding syllable.
(b) Syllables preceding a "ra-vattu" (ra-conjunct) in many poetic contexts.

A.2 *Prāsa* Equivalence Groups

Table 9 lists the three phonemic equivalence groups for *prāsa* consonant matching. All other consonants (approximately 30) form singleton equivalence classes.

Table 9: *Prāsa* consonant equivalence groups.

Group	Members (IAST)	Phonetic Basis
1	{la, ḷa}	Laterals
2	{śa, sa}	Sibilants
3	{ṛa, ra}	Rhotics

A.3 *Yati Maitri* Groups

Table 10 lists the 11 *maitri* equivalence groups for *yati* alliteration checking. Consonants and vowels within a group are treated as harmonically equivalent.

Table 10: *Yati maitri* groups (IAST romanisation).

#	Members (IAST)	Phonetic Basis
1	a, aa, ai, au, ha, ya, aṃ, ah	Open vowels + glides
2	i, ii, e, ee, ṛ	Front vowels
3	u, uu, o, oo	Back vowels
4	ka, kha, ga, gha, kṣa	Velars
5	ca, cha, ja, jha, śa, ṣa, sa	Palatals + sibilants
6	ṭa, ṭha, ḍa, ḍha	Retroflexes
7	ta, tha, da, dha	Dentals
8	pa, pha, ba, bha, va	Labials
9	ra, la, ṛa, ḷa	Liquids
10	na, ṇa	Nasals
11	ma	Labial nasal

B FST Pipeline Details

B.1 Codepoint Classifier Output Categories

Table 11 describes the 11 character categories output by Stage 1 of the FST pipeline.

Table 11: Codepoint classifier output categories (Telugu Unicode block, U+0C00–U+0C7F).

Category	Sym.	Range	Description
CONSONANT	C	U+0C15–39	ka to Ra
INDEP. VOWEL	V	U+0C05–14	a to au
MATRA	M	U+0C3E–4C	Vowel signs
VIRAMA	V⁻	U+0C4D	Halant
ANUSVARA	A	U+0C02	ṃ
VISARGA	VS	U+0C03	h
AI LENGTH	AL	U+0C56	ai-length mark
SPACE	SP	U+0020	Space
NEWLINE	NL	U+000A	Newline
ZWNJ	ZW	U+200C	Zero-width NJ
OTHER	OT	(other)	Digits, punctuation

B.2 Guru/Laghu Classifier FST Transition Table

Table 12 gives the complete transition function of Stage 3. S denotes an incoming syllable; P is the buffered pending syllable; $S2$ is the lookahead syllable. PEND. CLEAR and PEND. BOUNDARY abbreviate PENDING_CLEAR and PENDING_BOUNDARY.

Table 12: Guru/Laghu classifier FST transition table.

From	Input	Action	Next
IDLE	syllable S	buffer S; compute self_class(S)	Pend. Clear
IDLE	space / NL	emit boundary	IDLE
Pend. Clear	syllable $S2$	if conjunct: emit U (R5); else emit self_class(P); buffer $S2$	Pend. Clear
Pend. Clear	space	set boundary flag	Pend. Boundary
Pend. Boundary	syllable $S2$	emit self_class(P); emit boundaries; buffer $S2$	Pend. Clear
Pend. Boundary	space / NL	buffer boundary	Pend. Boundary
Pend. Clear	FLUSH	emit self_class(P)	IDLE
Pend. Boundary	FLUSH	emit self_class(P); emit boundaries	IDLE

B.3 State Budget Table

Table 13 gives the full per-component state count for the Dvipada constrained-decoding system summarised in Section 7.

Component	Type	States
Codepoint Classifier	Stateless	0 (inlined)
Syllable Assembler	Mealy FST	4
Guru/Laghu Classifier	Mealy FST	2
Position Tracker	Controller	3 (counter)
Gaṇa NFA	NFA	69
Prāsa NFA	NFA	7
Yati NFA	NFA	4
Total (separate)		**∼80**
Product (equiv.)	DFA	1,932

Table 13: State budget for the constrained decoding system.

B.4 Symbolic Enforcer Pipeline Diagram

Figure 1 shows the full Symbolic Enforcer pipeline summarised in Section 7.

C Detailed Validation Results

C.1 Length Ratio Distribution

Table 14 reports the full distribution of length ratios $r = |P|/|V|$ between Telugu prose meaning and poetic verse.

Table 14: Length ratio distribution ($n = 27{,}881$). Mean ratio: 1.63; median: 1.61. Enforced bounds: $0.8 \leq r \leq 2.5$.

Ratio Range	Count	Share
0.50 – 0.80	1	<0.1%
0.80 – 1.00	78	0.3%
1.00 – 1.50	9,333	33.5%
1.50 – 2.00	15,449	55.4%
2.00 – 5.11	3,020	10.8%
Pass rate	**99.1% (27,633 / 27,881)**	

C.2 Extended Embedding Model Results

Tables 15 and 16 provide full per-pair results for mSBERT-mpnet (Reimers and Gurevych, 2020) and L3Cube-IndicSBERT (Deode et al., 2023) respectively.

Table 15: mSBERT-mpnet results across all text pairs ($n = 29{,}343$).

Pair	Pass%	Mean	Median
Poem – Telugu meaning	63.7	0.685	0.701
Poem – English meaning	4.4	0.424	0.422
Telugu – English	23.7	0.501	0.515

Table 16: L3Cube-IndicSBERT results across all text pairs ($n = 29{,}343$).

Pair	Pass%	Mean	Median
Poem – Telugu meaning	40.1	0.614	0.618
Poem – English meaning	0.0	0.255	0.251
Telugu – English	21.0	0.579	0.585

C.3 LaBSE Similarity Distribution

Table 17 gives the full distribution of LaBSE Telugu-to-English cosine similarities.

Table 17: LaBSE Telugu meaning – English meaning similarity distribution ($n = 29{,}343$).

Range	Count	Share
0.00 – 0.30	3	<0.1%
0.30 – 0.50	592	2.0%
0.50 – 0.70	10,620	36.2%
0.70 – 0.85	16,294	55.5%
0.85 – 1.00	1,834	6.3%

C.4 Lexical Diversity Metrics (Full)

Table 18 reports the complete corpus-level and per-poem lexical-diversity metrics summarised inline in Section 5.

Metric	Scope	Value	Reference
TTR (V/N)	Corpus	0.491	Templin (1957)
MATTR (w=50)	Corpus	0.501	Covington and McFall (2010)
Yule's K	Corpus	3.497	Yule (1944)
Honoré's H	Corpus	6164.7	Honoré (1979)
Hapax ratio (V_1/V)	Corpus	0.804	–
Sichel's S (V_2/V)	Corpus	0.091	Sichel (1975)
Per-poem TTR (mean)	Poem	0.993	–
Per-poem MATTR (w=2, mean)	Poem	1.000	–
Per-poem Yule's K (mean)	Poem	22.74	–
Per-poem Honoré's H (mean)	Poem	2356.4	–
Gemma 3 Telugu coverage	Subword	74.4%	–
Per-poem subword TTR (mean)	Subword	0.897	–
Exact duplicates	Corpus	0	–
Near duplicates	Corpus	160 (79 grp)	–

Table 18: Lexical diversity metrics for the validated Dvipada corpus (27,881 poems, 179,812 tokens, 88,295 types).

C.5 LLM-as-a-Judge Per-Dimension Breakdown

Table 19 reports the per-dimension means and rating distribution for the LLM-as-a-Judge evaluation summarised inline in Section 5.

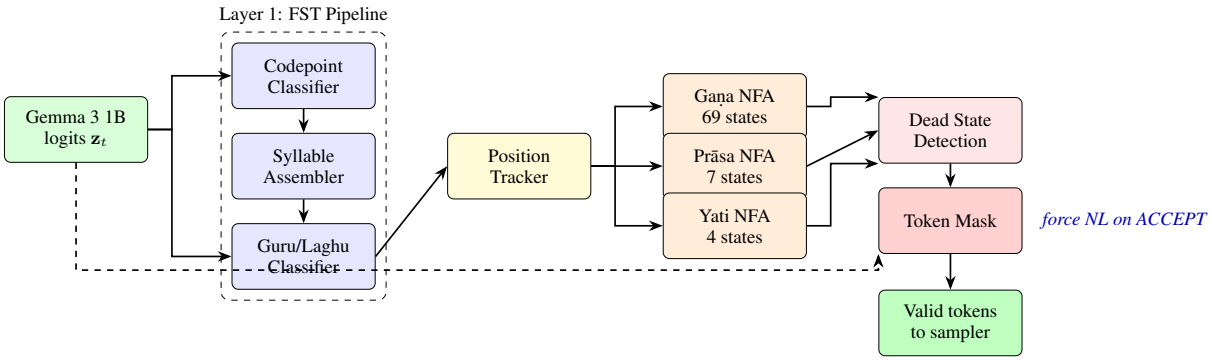

Figure 1: Symbolic Enforcer pipeline. Candidate tokens from Gemma 3 1B pass through a two-stage FST pipeline (with inlined codepoint classifier), are routed by the Position Tracker to three parallel NFAs (Gaṇa, Prāsa, Yati), and receive a boolean mask ($-\infty$ for invalid tokens) via dead state detection. Dashed arrow: raw logits bypass path to the mask.

Table 19: LLM-as-a-Judge evaluation ($n = 250$). Columns give percentage of samples at each rating. Mean total score: 24.25/25 (97.0%).

Dimension	Mean	5	4	3	≤ 2
Semantic Fidelity	4.73	80.0	14.4	4.0	1.6
Completeness	4.76	80.8	15.6	2.8	0.8
Cultural Accuracy	4.89	93.6	2.8	2.4	1.2
Telugu Linguistic	4.98	98.0	1.6	0.4	0.0
English Linguistic	4.90	90.4	8.8	0.8	0.0

Table 21: LLM-as-a-Judge stratified sample source distribution.

Source	Samples
Ranganatha Ramayanamu	42
Basavapuranamu	42
Dvipada Bhagavatamu 2	42
Srirama Parinayamu	42
Palanati Vira Charitra	41
Synthetic (LLM)	41
Total	**250**

C.6 LLM-as-a-Judge Aggregate Score Distribution

Table 20 gives the full distribution of aggregate scores (out of 25). Table 21 gives the stratified source breakdown.

Table 20: LLM-as-a-Judge aggregate score distribution ($n = 250$). Mean: 24.25/25 (97.0%).

Score / 25	Count	Share
25	172	68.8%
24	33	13.2%
23	22	8.8%
22	8	3.2%
21	6	2.4%
20	1	0.4%
19	2	0.8%
18	4	1.6%
17	1	0.4%
16	1	0.4%

D Detailed End-to-End Benchmark Results

This appendix provides per-topic accuracy and per-poem efficiency statistics for the twelve constrained-decoding configurations summarised in Section 9 (Tables 6 and 7). All numbers are extracted directly from the benchmark JSON files distributed with the project repository.

D.1 Per-topic Dvipada accuracy

Table 22 reports the number of valid poems out of 34 seeds for each Telugu prompt. T1 = "*tallI prEma annintikaMTe goppadi*" (mother's love), T2 = "*hanumaMtuDu samudramunu dATi laMkanu cErenu*" (Hanuman crosses to Lanka), T3 = "*SrIrAmuDu sItAdEvini rakshiMcuTaku laMkaku veLLenu*" (Rama goes to Lanka). Each cell shows valid/34 (p%).

Table 22: Per-topic poem accuracy on Dvipada (3 prompts × 34 seeds = 102 poems per row). The adapted Chandomitra port (Jagadeeshan et al., 2026) is included as an external baseline.

Model	Method	T1	T2	T3
Adapted Chandomitra baseline				
gemma3-1b-base	chandomitra	0/34 (0.0)	0/34 (0.0)	1/34 (2.9)
gemma3-1b-merged	chandomitra	2/34 (5.9)	4/34 (11.8)	8/34 (23.5)
FST+NFA Symbolic Enforcer				
gemma3-1b-base	masking_only	11/34 (32.4)	14/34 (41.2)	7/34 (20.6)
gemma3-1b-base	masking_backtrack	32/34 (94.1)	31/34 (91.2)	30/34 (88.2)
gemma3-1b-base	hybrid	34/34 (100.0)	34/34 (100.0)	34/34 (100.0)
gemma3-1b-merged	masking_only	12/34 (35.3)	9/34 (26.5)	16/34 (47.1)
gemma3-1b-merged	masking_backtrack	27/34 (79.4)	28/34 (82.4)	25/34 (73.5)
gemma3-1b-merged	hybrid	34/34 (100.0)	34/34 (100.0)	34/34 (100.0)

D.2 Per-topic Ragale accuracy

Table 23 reports the same breakdown for the three Kannada topics.

Table 23: Per-topic poem accuracy on Ragale (3 prompts × 34 seeds = 102 poems per row).

Model	Method	Bubbles	Moon	Rain
gemma3-1b-base	masking_only	4/34 (11.8)	4/34 (11.8)	5/34 (14.7)
gemma3-1b-base	masking_backtrack	4/34 (11.8)	4/34 (11.8)	5/34 (14.7)
gemma3-1b-base	hybrid	17/34 (50.0)	24/34 (70.6)	28/34 (82.4)
gemma3-1b-lora	masking_only	33/34 (97.1)	34/34 (100.0)	31/34 (91.2)
gemma3-1b-lora	masking_backtrack	33/34 (97.1)	34/34 (100.0)	31/34 (91.2)
gemma3-1b-lora	hybrid	34/34 (100.0)	34/34 (100.0)	34/34 (100.0)

D.3 Per-poem efficiency statistics

Table 24 reports the per-poem cost profile across all twelve configurations: average tokens generated, average mask computations, average backtracks, and total wall time over 102 poems.

Table 24: Per-poem efficiency profile across the twelve benchmark configurations and the adapted Chandomitra baseline. Tok = mean tokens generated; Mask = mean dynamic NFA mask computations (n/a for the adapted Chandomitra port, which uses a prefix-trie filter and does not maintain NFA state); BT = mean backtrack count; Total = total wall time over 102 poems.

Lang	Model	Method	Tok	Mask	BT	Total (s)
Dvipada	gemma3-1b-base	chandomitra	74.0	–	0.00	3865.5
Dvipada	gemma3-1b-merged	chandomitra	97.3	–	0.00	6071.9
Dvipada	gemma3-1b-base	masking_only	52.1	50.2	0.00	355.7
Dvipada	gemma3-1b-base	masking_backtrack	34.0	49.1	7.10	352.1
Dvipada	gemma3-1b-base	hybrid	29.9	27.9	0.01	202.9
Dvipada	gemma3-1b-merged	masking_only	44.9	38.9	0.00	214.4
Dvipada	gemma3-1b-merged	masking_backtrack	38.9	54.5	9.06	270.4
Dvipada	gemma3-1b-merged	hybrid	34.0	28.0	0.00	156.0
Ragale	gemma3-1b-base	masking_only	77.2	75.7	0.00	1092.9
Ragale	gemma3-1b-base	masking_backtrack	77.2	75.7	0.00	386.0
Ragale	gemma3-1b-base	hybrid	31.1	29.2	0.01	136.0
Ragale	gemma3-1b-lora	masking_only	25.8	23.8	0.00	145.2
Ragale	gemma3-1b-lora	masking_backtrack	25.8	23.8	0.00	145.7
Ragale	gemma3-1b-lora	hybrid	25.7	23.7	0.00	207.1

D.4 Methodology notes

Each row in the tables above corresponds to a single benchmark script invocation. Seeds follow the schedule $42 + 7k$ for $k \in [0, 33]$ to guarantee reproducibility while spanning the full RNG space the sampler explores. The validator is the same chandas pipeline (FST + parallel NFAs) used during corpus construction in Section 5, so a poem labelled *valid* here would also pass the inclusion criterion that gated entry into the training corpus. Prompt templates differ between the base and fine-tuned models (English instruction template for LoRA models, Kannada/Telugu system-prompt template for base models) so that each model is queried in the format closest to its training distribution; both templates are described in Section 7. The Ragale base model is the only configuration where the same script run twice (masking_only vs masking_backtrack) produces identical *validity* numbers but different *wall times*, because backtracking can escape doomed sequences earlier even when no alternative valid completion exists.

E Kannada Script Conversion Details

E.1 Character Coverage

Table 25 summarises the bidirectional converter's character coverage by category, based on Nidamarthy (2021).

Table 25: Kannada-Telugu bidirectional converter character coverage (Nidamarthy, 2021).

Category	Count	Resemblance
Consonants (high)	17	99%
Consonants (medium)	5	70–99%
Consonants (low)	5	<70%
Remaining consonants	8	(varies)
Vowels + signs	33	95–99%
Numerals	10	1:1 mapping
Total	**78+**	

E.2 Output Schema

Each entry in the converted Kannada dataset contains: the original Telugu poem; the converted Kannada poem; Telugu and Kannada prose meanings; the English meaning (unchanged); word-to-word glosses in Kannada script; a conversion quality score; and per-entry counts of high-, medium-, and low-resemblance characters used to compute the quality score.

E.3 Limitations

The conversion operates purely at the character level and cannot handle script-specific ligature conventions or rendering differences between the two scripts. Kannada-specific prosody rules are not yet validated; the converter assumes that *dvipada*

metrical rules transfer identically from Telugu to Kannada, which holds for shared Dravidian metres but would require independent verification for Kannada-specific forms. Converted prose meanings may sound unnatural when read as Kannada, though they function adequately as conditioning prompts for generation.

Repo: https://github.com/samvarankashyap/indicnuerosym

F Constrained Decoding Algorithms

This appendix presents the formal pseudocode for the dead state detection mechanism (Algorithm 1), the stateless analysis subroutine (Algorithm 2), the dynamic mask builder (Algorithm 3), and the decoding strategies referenced in Section 7: a stateless *Rejection Sampling* variant (Algorithm 4, reference only, not benchmarked), *Masking with Backtracking* (Algorithm 5), and *Hybrid Mask + Accept-State Re-ranking* (Algorithm 6). The benchmarked *Masking-Only* strategy is Algorithm 5 run with the backtracking block disabled and is not shown separately.

Notation. θ: language model; $\mathcal{T}$: tokenizer; $\mathbf{z}$: logit vector; $\boldsymbol{\pi}$: probability distribution; τ: temperature; $\mathbf{y}$: generated token sequence; $\mathcal{S}$: CompositeState (incremental FST+NFA); $\mathcal{B}$: active NFA branches; $\mathcal{V}_{\text{tel}}$: pre-extracted Telugu token set; $\mathbf{M}_s$: static mask (0 for Telugu/EOS, $-\infty$ elsewhere); ℓ: lines completed; $\mathcal{C}$: checkpoint ring buffer; b, b_c: total and consecutive backtrack counters.

Algorithm 1 IsReachable: Dead State Detection

Require: $\mathcal{B}$: active NFA branches; n: syllables consumed in current line
Ensure: **true** iff any branch can reach Accept with $n_{\text{total}} \in [11, 15]$

```
for (s, g, j, M) ∈ B do
    if s = Accept then
        if n ∈ {11, ..., 15} then return true; continue
    end if
    r ← |Pool(s)[g]| − j                    ▷ symbols left in current gaṇa
    if s ≤ 2 then                           ▷ Indra slot
        ℓ_o ← r + (2−s) · 3 + 2             ▷ min to accept
        h ← r + (2−s) · 4 + 3               ▷ max to accept
    else                                    ▷ Surya slot (s = 3)
        ℓ_o ← r; h ← r
    end if
    if n + ℓ_o ≤ 15 and n + h ≥ 11 then
        return true
    end if
end for
return false                                ▷ all branches doomed
```

Algorithm 2 AnalyzeText: Stateless FST+NFA Analysis

Require: Unicode string w (prefix-stripped)
Ensure: (alive, ℓ_c, acc, n, $\mathcal{B}$)

```
Init fresh SyllAssembler A, GuruLaghuClassifier G
B ← SpawnSlot(0, ()); ℓ_c ← 0; mk ← []
for item ∈ A.process(w) do
    if item = "\n" then
        Flush G; advance B with labels
        if ∃ (Accept, *, *, *) ∈ B then
            ℓ_c += 1
        end if
        B ← SpawnSlot(0, ()); mk ← []
    else if item = "␣" then
        Flush boundary; advance B
    else
        Feed syllable to G; advance B
    end if
end for
Flush remaining; advance B
n ← |mk|
acc ← ∃ (Accept, *, *, *) ∈ B ∧ n ∈ {11...15}
alive ← IsReachable(B, n) ∨ (ℓ_c ≥ 2)
return (alive, ℓ_c, acc, n, B)
```

Algorithm 3 BuildGanaMask: Dynamic NFA Token Mask

Require: σ: CompositeState snapshot; $\mathcal{V}_{\text{tel}} = (\text{ids}, \text{texts})$
Ensure: $\mathcal{V}_v$: token IDs keeping all NFAs alive

```
V_v ← ∅
for (tid, txt) ∈ zip(ids, texts) do
    S′ ← FromSnapshot(σ)
    S′.feed_token(txt); S′.flush()
    if S′.is_alive() then                   ▷ gaṇa ∧ prāsa ∧ yati
        V_v ← V_v ∪ {tid}
    end if
end for
return V_v
```

Algorithm 4 Strategy 1: Rejection Sampling

```
Require: θ, 𝒯, τ, K, p
Ensure: Two-line Dvipada poem text
y ← [];  ℓ ← 0
while ℓ < 2 do
    π ← softmax(θ(y)/τ);  sort desc
    st ← AnalyzeText(StripPrefix(𝒯.dec(y)))
    ℓ ← st.ℓ_c;  if ℓ ≥ 2 then break
                                        ▷ Force newline on completion/overshoot
    if st.acc ∧ st.n ∈ {11...15} then
        y ← y ‖ [NL];  continue
    end if
    if st.n > 15 ∧ ¬st.acc then
        y ← y ‖ [NL];  continue
    end if
    K′ ← K;  if st.n ≥ 8 then K′ ← 2K
                                        ▷ Rejection loop over top-K′
    V_acc ← ∅;  V_alv ← ∅
    for k = 1 to K′ do
        c ← π_k;  if π(c) < 10^−8 then break
        if c = Eos ∧ ℓ ≥ 2 then select c; break
        if ¬IsTelugu(𝒯.dec(c)) then continue
        r ← AnalyzeText(StripPrefix(𝒯.dec(y ‖ [c])))    ▷ full re-analysis
        if ¬r.alive then continue
        if "\n" ∈ 𝒯.dec(c) ∧ r.ℓ_c ≤ ℓ then continue
        if r.n > 15 ∧ ¬r.acc then continue
        if r.acc ∧ r.n ∈ {11...15} then
            V_acc ← V_acc ∪ {(c, π(c))}
        end if
        V_alv ← V_alv ∪ {(c, π(c))}
        if |V_acc| ≥ 5 then break
    end for
                                        ▷ Select from best candidate set
    V ← V_acc if ≠ ∅, else V_alv
    if V ≠ ∅ then
        Top-p filter;  chosen ← weighted sample from V
    else
                                        ▷ Exhaustive fallback (500 tokens)
        for k = 1 to 500 do
            if IsTelugu(𝒯.dec(π_k)) then
                r ← AnalyzeText(StripPrefix(𝒯.dec(y ‖ [π_k])))
                if r.alive ∧ (r.n ≤ 15 ∨ r.acc) then
                    chosen ← π_k;  break
                end if
            end if
        end for
    end if
    y ← y ‖ [chosen]
end while
return 𝒯.dec(y)
```

Algorithm 5 Strategy 2: Logit Masking with Backtracking

```
Require: θ, 𝒯, V_tel, M_s, τ, τ_max=1.2, Δτ=0.15, D=6
Ensure: Two-line Dvipada poem text
y ← [];  𝒮 ← new CompositeState();  ℓ ← 0;  τ_c ← τ
𝒞 ← [];  b ← 0;  b_c ← 0;  d ← 0                ▷ ring buffer, counters
for t = 1 to T_max do
    if ℓ ≥ 2 then break
    end if
    z ← θ(y)/τ_c + M_s                          ▷ static mask
                                        ▷ Checkpoint every 3 tokens
    if t mod 3 = 0 then
        𝒞.push(t, 𝒮.snap(), |y|, ℓ, τ_c)
        if |𝒞| > 15 then 𝒞 ← 𝒞[−15 :]
    end if
                                        ▷ Force newline on ACCEPT
    𝒮_f ← clone(𝒮);  𝒮_f.flush()
    if 𝒮_f.has_accept() then
        𝒮.feed("\n");  ℓ ← 𝒮.ℓ_c;  y ← y ‖ [NL]
        b_c ← 0;  continue
    end if
                                        ▷ Build dynamic NFA mask
    σ ← 𝒮.snap();  V_v ← BuildGanaMask(σ, V_tel)
                                        ▷ Filter spurious newlines
    if NL ∈ V_v then
        𝒮′ ← FromSnap(σ);  𝒮′.feed("\n")
        if 𝒮′.ℓ_c ≤ ℓ then
            V_v ← V_v \ {NL}
        end if
    end if
                                        ▷ Backtrack if dead end
    if (|V_v| < 3 ∨ 𝒮_f.n > 15) and |𝒞| > 0 and b < 30 then
        b += 1;  b_c += 1
        Pop min(b_c, |𝒞|) from 𝒞
        Restore 𝒮, y, ℓ, t from last popped
        τ_c ← min(ckpt.τ + Δτ · b_c, τ_max)         ▷ escalate
        d ← D;  Seed(seed + b · 1337);  continue
    end if
                                        ▷ Apply dynamic mask and sample
    M_d[i] ← 0 if i ∈ V_v, else −∞
    z ← z + M_d;  π ← softmax(z)
    chosen ← sample(π, p)
                                        ▷ Commit token
    y ← y ‖ [chosen];  𝒮.feed_token(𝒯.dec(chosen))
    ℓ ← 𝒮.ℓ_c;  b_c ← 0
                                        ▷ Temperature decay
    if d > 0 then
        d −= 1;  if d = 0 then τ_c ← τ
    end if
end for
return 𝒯.dec(y)
```

Algorithm 6 Strategy 3: Hybrid (Mask + Accept-State Re-ranking)

Require: $\theta, \mathcal{T}, \mathcal{V}_{\text{tel}}, \mathbf{M}_s, \tau, K_r{=}100$
Ensure: Two-line Dvipada poem text

$\mathbf{y} \leftarrow [\,]$; $\mathcal{S} \leftarrow$ new CompositeState(); $\ell \leftarrow 0$; $\tau_c \leftarrow \tau$
$\mathcal{C} \leftarrow [\,]$; $b \leftarrow 0$; $b_c \leftarrow 0$
for $t = 1$ **to** $T_{\max}$ **do**
  if $\ell \geq 2$ **then break**
  end if
  $\mathbf{z} \leftarrow \theta(\mathbf{y})/\tau_c + \mathbf{M}_s$
  if $t \bmod 3 = 0$ **then** checkpoint $\mathcal{C}$ (cap 15)
  ▷ *Force newline on ACCEPT*
  $\mathcal{S}_f \leftarrow$ clone($\mathcal{S}$); $\mathcal{S}_f$.flush()
  if $\mathcal{S}_f$.has_accept() **then**
    $\mathcal{S}$.feed("\n"); $\ell \leftarrow \mathcal{S}.\ell_c$; $\mathbf{y} \leftarrow \mathbf{y} \,\|\, [\text{NL}]$; $b_c \leftarrow 0$; **continue**
  end if
  ▷ *Build + apply dynamic NFA mask*
  $\sigma \leftarrow \mathcal{S}$.snap(); $\mathcal{V}_v \leftarrow$ BuildGanaMask($\sigma, \mathcal{V}_{\text{tel}}$)
  Filter spurious newlines (as Alg. 5)
  $\mathbf{M}_d[i] \leftarrow 0$ if $i \in \mathcal{V}_v$, else $-\infty$; $\mathbf{z} \leftarrow \mathbf{z} + \mathbf{M}_d$
  $\boldsymbol{\pi} \leftarrow$ softmax($\mathbf{z}$); sort desc
  ▷ *Accept-state re-ranking pass over top-K_r from masked dist.*
  $\mathcal{V}_{\text{acc}} \leftarrow \emptyset$; $\mathcal{V}_{\text{alv}} \leftarrow \emptyset$
  for $k = 1$ **to** K_r **do**
    $c \leftarrow \pi_k$; **if** $\boldsymbol{\pi}(c) < 10^{-8}$ **then break**
    if $c = \text{Eos} \wedge \ell \geq 2$ **then** select c; **break**
    ▷ Clone state, simulate candidate
    $\mathcal{S}' \leftarrow$ FromSnap(σ)
    for each ch $\in \mathcal{T}$.dec(c) **do** $\mathcal{S}'$.feed(ch)
    $\mathcal{S}'_f \leftarrow$ clone($\mathcal{S}'$); $\mathcal{S}'_f$.flush()
    if $\neg\mathcal{S}'_f$.alive() **then continue**
    if "\n" $\in \mathcal{T}$.dec(c) $\wedge\, \mathcal{S}'.\ell_c \leq \ell$ **then continue**
    if $\mathcal{S}'_f.n > 15 \wedge \neg\mathcal{S}'_f$.acc() **then continue**
    if $\mathcal{S}'_f$.has_accept() **then**
      $\mathcal{V}_{\text{acc}} \leftarrow \mathcal{V}_{\text{acc}} \cup \{(c, \boldsymbol{\pi}(c))\}$
    end if
    $\mathcal{V}_{\text{alv}} \leftarrow \mathcal{V}_{\text{alv}} \cup \{(c, \boldsymbol{\pi}(c))\}$
    if $|\mathcal{V}_{\text{acc}}| \geq 3$ **then break**
    if $|\mathcal{V}_{\text{alv}}| \geq 10 \wedge \mathcal{V}_{\text{acc}} = \emptyset$ **then break**
  end for
  ▷ *Select or backtrack*
  $\mathcal{V} \leftarrow \mathcal{V}_{\text{acc}}$ if $\neq \emptyset$, else $\mathcal{V}_{\text{alv}}$
  if $\mathcal{V} \neq \emptyset$ **then**
    Normalize; chosen $\leftarrow$ weighted sample
  else if $|\mathcal{C}| > 0 \wedge b < 30$ **then**
    $b \mathrel{+}= 1$; $b_c \mathrel{+}= 1$; pop $\min(b_c, |\mathcal{C}|)$ from $\mathcal{C}$
    Restore $\mathcal{S}, \mathbf{y}, \ell, t$; $\tau_c \leftarrow \min(\text{ckpt}.\tau + \Delta\tau \cdot b_c,\ \tau_{\max})$
    Seed(seed$+b\cdot$1337); **continue**
  else
    chosen $\leftarrow \pi_1$ ▷ last resort
  end if
  $\mathbf{y} \leftarrow \mathbf{y} \,\|\, [\text{chosen}]$; $\mathcal{S}$.feed_token($\mathcal{T}$.dec(chosen)); $\ell \leftarrow \mathcal{S}.\ell_c$; $b_c \leftarrow 0$
end for
return $\mathcal{T}$.dec($\mathbf{y}$)

Table 26: Comparison of constrained decoding strategies. The first column (Stateless Rejection) is the reference variant of Algorithm 4 (not benchmarked in Section 9); the last two columns correspond to the benchmarked Masking+Backtracking and Hybrid strategies. All three share the same NFA machinery but differ in how the NFAs are consulted per step.

Property	Stateless Rej. (Alg 4)	Mask. + BT (Alg 5)	Hybrid (Alg 6)
State track.	Stateless	Incremental	Incremental
Per-step	$O(KL)$	$O(\lvert\mathcal{V}_t\rvert)$	$O(\lvert\mathcal{V}_t\rvert + K_r)$
Seq. dep.	Yes	No	No
Reject. set	Top-K	All Telugu	All Telugu
Recovery	Exhaust.	Backtrack	Backtrack
Temp. adapt.	None	Escalation	Escalation
Constraints	Gaṇa reach.	All 3 (alive)	All 3 (alive)
Accept re-rank	✓ (first hit)	×	✓ (top-100)

G Ragale Constrained Decoding: Construction Details

This appendix provides full construction details for the Kannada *utsaha ragale* variant of the Symbolic Enforcer summarised in Section 7.8. The FST pipeline (Syllable Assembler and Guru/Laghu Classifier) is architecturally identical to the Dvipada pipeline; only the Unicode codepoint sets change from Telugu (U+0C00–U+0C7F) to Kannada (U+0C80–U+0CFF). The five guru/laghu classification rules (Section 7.3) apply unchanged.

G.1 Structural Differences from Dvipada

Table 27 summarises the key differences between the two metres' constraint specifications.

Table 27: Dvipada vs. Ragale constraint comparison.

Aspect	Dvipada (Telugu)	Ragale (Kannada)
Gaṇa patterns	8 (6 Indra + 2 Surya)	2 (III, IIU)
Formal language	$(6\text{ pat.})^3 \cdot (2\text{ pat.})$	$(\text{III} \mid \text{IIU})^3 \cdot \text{IIU}$
Syllables/line	11–15 (variable)	12 (fixed)
Gaṇa sizes	2–4 syllables	3 (uniform)
Prāsa	7 states, 3 equiv. groups	7 states, 2 equiv. groups
Yati	4 states (optional)	Not used
Guru ending	Implicit	Enforced (gaṇa 4 = IIU)

G.2 Gaṇa NFA for Ragale

The Ragale Gaṇa NFA validates each line against the regular pattern:

$$L_{\text{line}} = (\text{III} \mid \text{IIU})^3 \cdot \text{IIU} \tag{2}$$

Only two gaṇa patterns are permitted (Table 2): III (three *laghu*) at positions 1–3, and IIU (two *laghu* + one *guru*) at all four positions. Position 4 is constrained to IIU, structurally enforcing the *guru* ending rule. This yields exactly $2^3 = 8$ valid line patterns, all of length 12 syllables.

The NFA has **22 states**: slots 0–2 each spawn 2 candidate *gaṇas* with 3 sub-positions ($2 \times 3 \times 3 = 18$), slot 3 spawns only IIU ($1 \times 3 = 3$), plus 1 ACCEPT state. Compared to the Dvipada Gaṇa NFA (69 states), the Ragale variant achieves a $3.1\times$ reduction.

Dead state detection (Algorithm 1) simplifies because all *gaṇas* are exactly 3 syllables: $\mathit{min_dist}(s) = \mathit{max_dist}(s) = (3-j)+(3-s)\times 3$, where j is the sub-position and s is the slot index.

G.3 Prāsa and Guru Ending

The Prāsa NFA retains the same 7-state structure (LINE1_SYL0 through ACCEPT/REJECT) with two Kannada-specific equivalence groups: laterals

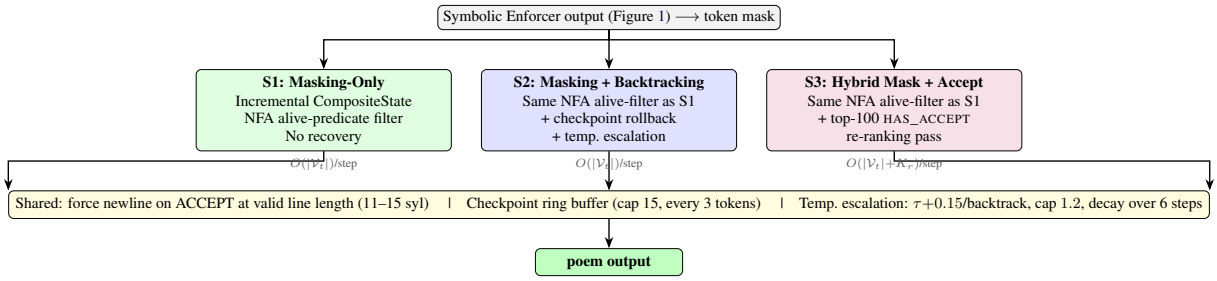

Figure 2: Three constrained decoding strategies using the Symbolic Enforcer. All three share the same per-step NFA alive-predicate filter via BUILDGANAMASK over the ∼1,822 Telugu tokens ($O(|\mathcal{V}_{\text{tel}}|)$ per step, sequence-length independent). (S1) Masking-only: no recovery, no re-ranking. (S2) Masking with backtracking: adds checkpoint rollback and temperature escalation when the alive set collapses. (S3) Hybrid: adds a second NFA pass over the top-K_r=100 masked candidates that re-ranks by HAS_ACCEPT (line-completion) preference; backtracking is retained only as a rare fallback.

{ḷa ↔ la} and sibilants {śa ↔ ṣa ↔ sa}. The rhotic group ({ṛa, ra}) present in Telugu is absent in Kannada, reducing from 3 to 2 equivalence pairs.

The *guru* ending constraint is not checked as a separate rule; it is structurally guaranteed by the Gaṇa NFA's restriction of position 4 to IIU, whose final syllable is always *guru* (U).

G.4 State Budget

Table 28 shows the Ragale state budget.

Table 28: State budget for the Ragale constrained decoding system.

Component	Type	States
Codepoint Classifier	Stateless	0 (inlined)
Syllable Assembler	Mealy FST	4
Guru/Laghu Classifier	Mealy FST	2
Position Tracker	Controller	3 (counter)
Gaṇa NFA	NFA	22
Prāsa NFA	NFA	7
Yati NFA	–	0 (not used)
Total (separate)		**35**
Product (equiv.)	DFA	154

The total state count is 35 (2.3× fewer than Dvipada's ∼80). The product automaton is 22 × 7 = 154 states (12.5× fewer than Dvipada's 1,932), reflecting the reduced combinatorial complexity of the 2-pattern gaṇa system. Using separate NFAs still provides a 4.4× reduction over the product (35 vs. 154).

G.5 Decoding Strategies

All three decoding strategies from Section 7 apply to Ragale without algorithmic modification:

- **Rejection sampling** (Algorithm 4): ANALYZETEXT (Algorithm 2) runs the Kannada FST pipeline with VALID_LINE_LENGTHS = {12}.
- **Logit masking with backtracking** (Algorithm 5): BUILDGANAMASK (Algorithm 3) tests each Kannada token against the Ragale COMPOSITESTATE, which checks gaṇa reachability and prāsa validity (no yati check).
- **Hybrid** (Algorithm 6): identical mask-then-reject loop.

The only implementation change is the reduced gaṇa pool (2 patterns vs. 8) and the removal of the yati alive check from COMPOSITESTATE.IS_ALIVE(). Dead state detection (Algorithm 1) uses simplified closed-form bounds where *min_dist* = *max_dist* (all *gaṇas* are 3 syllables). At runtime, the smaller NFA produces fewer branches, reducing the per-step mask computation cost.

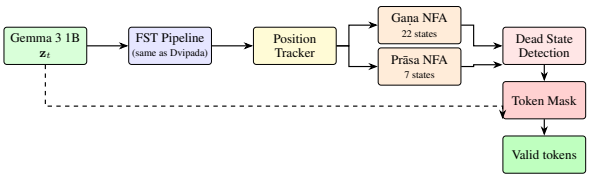

Figure 3: Ragale Symbolic Enforcer. Compared to the Dvipada pipeline (Figure 1), Yati NFA is removed and the Gaṇa NFA uses 22 states (2 patterns) instead of 69 (8 patterns). The FST pipeline is identical except for Kannada Unicode codepoints.

H Generation Examples

This appendix presents paired success and failure examples for the five most important cells of

Tables 6 and 7, plus the adapted Chandomitra baseline. Each example shows the generated couplet in RTS romanisation (matching the prompt convention of Section 8), the FST-emitted U/I (*guru*/*laghu*) syllable-weight markers, and a per-constraint verdict (✓ = pass, × = fail). Examples are selected deterministically as the lowest seed satisfying the filter (all-valid for successes, not-all-valid for failures); cells that are 100% valid use the lowest-seed record. Original Telugu and Kannada script for every example is available in the corresponding benchmark JSON file.

H.1 Dvipada (Telugu)

Hybrid, gemma3-1b-merged — success. Topic: *tallI prEma annintikaMTe goppadi* (mother's love).

> *liMgamaMtayu niMpeDi briyamu liMga*
> *liMgaMbunaMduna nellavArikini*
> U/I L1: U I U I I U I I I I I U I
> U/I L2: U U I U I I U I U I I I
> **Verdict:** *gaṇa* ✓, *prāsa* ✓, *yati* ✓
> seed=42, backtracks=0, tokens=35, elapsed=2.04 s

Masking-only, gemma3-1b-base — success. Topic: *tallI prEma annintikaMTe goppadi*. The base model collapses onto a degenerate but technically valid loop, repeating the same line verbatim — still passes all three constraints.

> *nati nUta nUta nUta nanUnakaina*
> *nati nUta nUta nUta nanUnakaina*
> U/I L1: I I U I U I U I I U I U I
> U/I L2: (identical to L1)
> **Verdict:** *gaṇa* ✓, *prāsa* ✓, *yati* ✓
> seed=42, backtracks=0, tokens=36, elapsed=2.70 s

Masking-only, gemma3-1b-base — failure. Topic: *SrIrAmuDu sItAdEvini rakshiMcuTaku laMkaku veLLenu* (Rama goes to Lanka). Line 1's foot partition is valid, but line 2 admits no legal Indra^3 · Surya partition over its 11 emitted syllables.

> *nalakala naMdena nalakala naMda*
> *sIlakunaMdena sItAmatini*
> U/I L1: I I I I U I I I I I I U I
> U/I L2: U I I U I I U U I I I
> **Verdict:** L1 *gaṇa* ✓; L2 *gaṇa* × (no valid foot partition).
> seed=42, backtracks=0, tokens=42, elapsed=2.80 s

H.2 Ragale (Kannada)

Hybrid, gemma3-1b-lora — success. Topic: *Bubbles*. Both lines reach the canonical III^3 · IIU partition with the structurally required *guru* ending.

> *kanasu muDiyu kadana hasivA*
> *olavanu kaDalanu madhuravA*
> U/I L1: I I I I I I I I I I U
> U/I L2: I I I I I I I I I I U
> **Verdict:** *gaṇa* ✓ (III III III IIU), *prāsa* ✓, *guru*-ending ✓
> seed=42, backtracks=0, tokens=27, elapsed=2.64 s

Masking-only, gemma3-1b-base — failure. Topic: *Bubbles*. Line 1 emits only 10 syllables (the metre requires exactly 12) and does not end on *guru*; line 2 happens to recover the canonical pattern. This illustrates the dominant Ragale-base failure mode: stateless masking lets the model under-shoot the line target.

> *ba cilA asakana as padya*
> *gaNanakana karaka karaka nE*
> U/I L1: I I U I I I I U U I (10 syllables)
> U/I L2: I I I I I I I I I I U (12 syllables)
> **Verdict:** L1 *gaṇa* × (10 syllables, expected 12), *guru*-ending ×; L2 *gaṇa* ✓.
> seed=42, backtracks=0, tokens=65, elapsed=9.81 s

H.3 Adapted Chandomitra Baseline

Chandomitra, gemma3-1b-merged — rare success. Topic: *hanumaMtuDu samudramunu dATi laMkanu cErenu* (Hanuman crosses to Lanka). One of only 14 valid records out of 102.

> *saraLamaina va bhASa jUpakuMDa livi*
> *baraga nanilajayapratimapadmamula*
> **Verdict:** *gaṇa* ✓, *prāsa* ✓, *yati* ✓ (per-constraint scores 100/100/100 from the chandomitra `breakdown` field).
> seed=49, backtracks=0, tokens=28, elapsed=0.64 s

Chandomitra, gemma3-1b-merged — failure. Topic: *tallI prEma annintikaMTe goppadi*. The most common adapted-Chandomitra failure mode: the model never emits a line break and the top-k filter exhausts the 150-token budget on a single

runaway line. This is the structural dead-state issue described in the §10 *Comparison to Chandomitra* paragraph.

> *saraLamaina vArakulu nirmANaMDabhuvaMTi vyeMDEgodhikaNa pramukhacE padavaMta ravidya cakra SiSEcElaM* . . .
> [150 tokens, no newline, no valid couplet emitted]
> **Verdict:** L1 *gaṇa* × (no foot partition over the runaway syllable stream); L2 **never emitted**.
> seed=42, backtracks=0, tokens=150, elapsed=104.35 s

Chandomitra, gemma3-1b-base — failure. Topic: *tallI prEma annintikaMTe goppadi*. A pedagogically useful failure: *gaṇa* passes on both lines, but *prāsa* fails (the second-syllable consonants m and r are not in the same equivalence group) and *yati* fails on line 1 (foot 1 first letter and foot 3 first letter belong to different *maitri* groups).

> *ammA dayakulE toli saha kar vyara Ta*
> *pada prativara mUDu mahavaccu lakSa*
> **Verdict:** *gaṇa* ✓/✓, *prāsa* × ($m \not\equiv r$), *yati* ×/✓ (L1 fails, L2 passes; aggregate yati score 50/100 in the chandomitra `breakdown`).
> seed=42, backtracks=0, tokens=23, elapsed=2.31 s